**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ,
СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

ОРДЕНА ТРУДОВОГО КРАСНОГО ЗНАМЕНИ
ФЕДЕРАЛЬНОГО ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

**«МОСКОВСКИЙ
ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
СВЯЗИ И ИНФОРМАТИКИ»**

(МТУСИ)

Кафедра «Корпоративные информационные системы»

ЛАБОРАТОРНАЯ РАБОТА № 4

по дисциплине «Прикладные информационные системы»
на тему:

СОЗДАНИЕ СТРАНИЦЫ ОПРЕДЕЛЕННОЙ ЗАПИСИ. ВЕРСТКА ОБЕИХ СТРАНИЦ В СООТВЕТСТВИИ С МАКЕТОМ.

Выполнила: студентка группы БСТ2256
Журавлева О.А.

Студенческий билет № 3БСТ22158

Проверил: Сергеев А.Е.

Москва 2026

С О Д Е Р Ж А Н И Е

Цель работы **3**

Задачи **3**

Ход работы **4**

1 Создание проекта **4**

1.1 Создание проекта и приложения 4

1.2 Настройка проекта 4

2 Создание страницы определённой записи **5**

2.1 Настройка URL с регулярным выражением 5

2.2 Создание представления get_article 5

2.3 Создание шаблона article.html 5

2.4 Модификация главной страницы архива 6

3 Верстка обеих страниц в соответствии с макетом **7**

3.1 Создание article.css и подключение CSS к шаблонам 7

3.2 Добавление уникальных классов body 7

3.3 Стилизация страниц 7

3.4 Проверка работы 9

Заключение **11**

Цель работы

Научиться создавать динамические страницы для отображения отдельной записи блога с использованием Django: настройка URL с параметрами, обработка исключений, создание шаблонов и стилизация страниц в соответствии с заданным макетом.

Задачи

1. Создать новый проект blog в папке lab4 и перенести в него приложение articles из предыдущей лабораторной работы.

2. Настроить URL для отображения страницы отдельной статьи с передачей идентификатора записи.

3. Реализовать представление get_article, обрабатывающее запрос и возвращающее страницу статьи или ошибку 404.

4. Создать шаблон article.html для отображения одной статьи.

5. Модифицировать шаблон archive.html, чтобы заголовки статей были ссылками на соответствующие страницы.

6. Подключить CSS-стили к обеим страницам.

7. Стилизовать страницы в соответствии с макетами lab4_main (архив) и lab4_item (отдельная статья), включая ссылку «Все записи» на странице статьи.

Ход работы

1 Создание проекта

1.1 Создание проекта и приложения

1. В папке lab4 выполним команду создания проекта:

```
django-admin startproject blog
```

2. Скопируем в проект папку articles из лабораторной работы.

1.2 Настройка проекта

1. В файле settings.py укажем базу данных:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': 'db_blog',
    }
}
```

2. Добавим приложение articles в INSTALLED_APPS:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'articles'
]
```

3. Выполним миграции для создания таблиц в консоли:

```
python manage.py migrate
```

4. Создадим суперпользователя для входа в административную панель:

```
python manage.py createsuperuser
```

2 Создание страницы определённой записи

2.1 Настройка URL с регулярным выражением

В файле urls.py добавим новый маршрут для отдельной статьи. Используем регулярное выражение с именованной группой article_id, которая принимает только цифры.

```
from django.contrib import admin
from django.urls import path, re_path
from articles.views import archive, get_article

urlpatterns = [
    path('', archive, name='archive'),
    re_path(r'^article/(?P<article_id>\d+)$', get_article, name='get_article'),
    path('admin/', admin.site.urls),
]
```

2.2 Создание представления get_article

1. В файле views.py добавим функцию get_article:

```
from django.shortcuts import render
from django.http import Http404
from .models import Article

def archive(request):
    posts = Article.objects.all()  # получаем все статьи из БД
    return render(request, 'archive.html', {'posts': posts})

def get_article(request, article_id):
    try:
        post = Article.objects.get(id=article_id)
        return render(request, 'article.html', {"post": post})
    except Article.DoesNotExist:
        raise Http404
```

article_id – параметр, переданный из URL.

Article.objects.get(id=article_id) – попытка найти статью с указанным идентификатором.

Если статья не найдена, возникает исключение Article.DoesNotExist, которое обрабатывается и вызывает Http404..

2.3 Создание шаблона article.html

В папке articles/templates/ создадим файл article.html:

```
{% load static %}
<!DOCTYPE html>
<html>
    <head>
        <title>{{ post.title }}</title>
        <link rel="stylesheet" href="{% static 'css/article.css' %}">
    </head>
    <body class="article">
        <div class="header">
            <img src="{% static 'images/logo.png' %}" alt="Логотип" class="logo" />
        </div>
        <div class="article">
            <h1 class="post-title">{{ post.title }}</h1>
            <div class="article-info">
                <div class="article-author">{{ post.author.username }}</div>
                <div class="article-created-date">{{ post.created_date }}</div>
            </div>
            <div class="article-text">{{ post.text|linebreaks }}</div>
            <div class="back-link">
                <a href="{% url 'archive' %}">Все записи</a>
            </div>
        </div>
    </body>
</html>
```

2.4 Модификация главной страницы архива

В шаблоне archive.html обернём заголовок каждой статьи в тег <a>, указывающий на страницу этой статьи:

```
{% load static %}
<!DOCTYPE html>
<html>
    <head>
        <title>Архив всех статей</title>
        <link rel="stylesheet" href="{% static 'css/article.css' %}">
    </head>
    <body class="archive">
        <div class="header">
            <img src="{% static 'images/logo.png' %}" alt="Логотип" class="logo" />
        </div>
        <div class="archive">
            {% for post in posts %}
            <div class="one-post">
                <h2 class="post-title">
                    <a href="{% url 'get_article' article_id=post.id %}">{{
post.title }}</a>
                </h2>
                <div class="article-info">
                    <div class="article-author">{{ post.author.username }}</div>
                    <div class="article-created-date">{{ post.created_date }}</div>
                </div>
                <p class="article-text">{{ post.get_excerpt }}</p>
            </div>
            {% endfor %}
        </div>
    </body>
</html>
```

3 Верстка обеих страниц в соответствии с макетом

3.1 Создание article.css и подключение CSS к шаблонам

1. В папке articles/static/css/ создадим файл article.css.

2. В шаблонах archive.html и article.html добавим в <head> ссылку на CSS:

```
    <head>
        <title>Архив всех статей</title>
        <link rel="stylesheet" href="{{ STATIC_URL }}css/article.css">
    </head>
```

3.2 Добавление уникальных классов body

1. В шаблоне archive.html добавим класс archive тегу <body>:

```
    <body class="archive">
```

2. В шаблоне article.html добавим класс article тегу <body>:

```
    <body class="article">
```

3.3 Стилизация страниц

1. В файле article.css зададим базовые стили, общие для обеих страниц:

```
body {
    background: #1abc9c;
    font-family: Tahoma, Arial, sans-serif;
    color: #ffffff;
}

.header img {
    display: block;
    width: 318px;
    margin-left: auto;
    margin-right: auto;
}

.archive, .article {
    width: 960px;
    margin-left: auto;
    margin-right: auto;
}
```

2. Стили для ссылок на заголовки статей:

```
.post-title a {
    color: #ffffff;
    text-decoration: none;
}

.post-title a:hover {
```

```
    text-decoration: underline;
}
```

3. Стили для блока с автором и датой (чтобы они находились на одной строке):

```
.article-author {
    width: 50%;
    float: left;
}

.article-created-date {
    text-align: right;
}

.article-info {
    overflow: hidden; /* чтобы обтекание не выпадало */
}
```

4. Дополнительные стили для отступов между статьями:

```
.one-post {
    margin-bottom: 30px;
}
```

5. Стили для заголовка статьи:

```
.article h1 {
    font-size: 28px;
    margin-bottom: 15px;
}
```

6. Стили для блока с информацией об авторе и дате:

```
.article .article-author {
    width: 50%;
    float: left;
}

.article .article-created-date {
    text-align: right;
}

.article .article-info {
    overflow: hidden;
    margin-bottom: 20px;
}
```

7. Стили для текста статьи:

```
.article-text {
    font-size: 18px;
    line-height: 1.4;
    margin-top: 20px;
}
```

8. Стили для ссылки «Все записи»:

```
.back-link {
    margin-top: 40px;
    text-align: center;
}

.back-link a {
    color: #ffffff;
    text-decoration: none;
    font-weight: bold;
}

.back-link a:hover {
    text-decoration: underline;
}
```

3.4 Проверка работы

1. Запустим сервер разработки и перейдём на главную страницу http://127.0.0.1:8000/ (рис. 1):

```
python manage.py runserver
```

Рисунок 1 – начальная страница.

Рисунок 2 – переход на страницу статьи Тест 1.

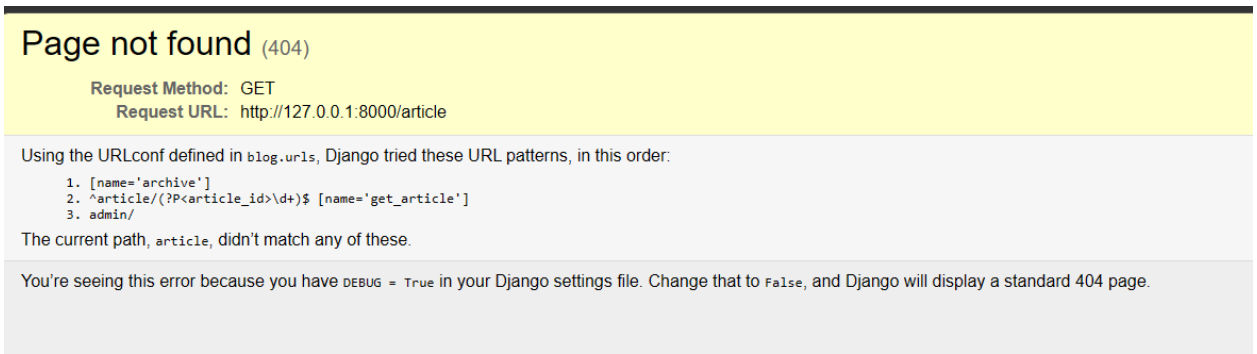

Рисунок 3 – переход на несуществующую страницу.

Заключение

В ходе лабораторной работы были выполнены все поставленные задачи: создан проект blog и перенесено приложение articles, настроен URL для отображения отдельной статьи с использованием регулярных выражений и именованной группы, реализовано представление get_article с обработкой исключения DoesNotExist и генерацией ошибки 404. Создан шаблон article.html для вывода одной статьи, а также модифицирован шаблон archive.html для превращения заголовков в ссылки. Подключён CSS-файл, добавлены уникальные классы тегам <body>, заданы общие и специфичные стили. Страницы стилизованы в соответствии с макетами: центрирование логотипа, ширина контента 960px, обтекание автора и даты, оформление ссылок, добавлена ссылка «Все записи» на странице статьи.